

A Security System for Health Facilities Medical Record Management in Kigali

Carnegie Mellon University Africa
Information Security, Spring 2026

Abstract. *This report presents MedVault, a security-focused overlay built for healthcare facilities in Kigali, Rwanda. The system addresses a documented gap in existing Electronic Medical Record deployments where technical safeguards do not meet current data-protection standards. Rather than replacing existing infrastructure, MedVault introduces a protection layer that enforces strong identity verification, field-level encryption, patient-controlled consent, and a tamper-evident audit trail. The report covers the system architecture, threat model, protocol design, implementation details, and a set of adversarial demonstrations that validate each security control.*

Christian, Raissa, Gustave

Contents

1	Introduction	2
2	System Architecture	2
2.1	Layer Structure	2
2.2	Security Pipeline	3
2.3	Data Separation	3
3	Threat Model	4
3.1	Assets	4
3.2	Adversary Classes	4
3.3	STRIDE Mapping	4
4	Protocol Design	4
4.1	Identity and Session Protocol	4
4.2	Replay Attack Prevention	5
4.3	Transport Security	5
4.4	Access Control and Consent	5
4.5	Encryption Protocol	6
4.6	Audit Chain Protocol	6
5	Implementation	6
5.1	Technology Stack	6
5.2	Security Components	6
5.3	Configuration and Secrets Management	7
5.4	Limitations	8
6	Attack and Defence Demonstrations	8
7	Conclusion	9

1 Introduction

This section is reserved for the authors. Write your own contextual motivation, problem statement, and contribution summary here. Suggested points: EMR adoption in Rwandan hospitals, the regulatory context (Law No. 058/2021 on personal data protection), the paper-digital boundary problem, and how MedVault addresses it.

Insert here: a photograph of a hospital reception or ward in Kigali showing paper records alongside a computer terminal, or a map of Rwanda highlighting healthcare facilities.

Figure 1: fig:intro

2 System Architecture

MedVault is built as a layered security overlay. It sits between client applications and the data store, enforcing security controls at each boundary. The design follows a principle of defence in depth: each layer operates independently, so a failure at one layer does not automatically compromise the layers below it.

2.1 Layer Structure

The system is organised into seven layers, shown in Figure 2. Requests enter at the top and must pass every layer before reaching the database. The first two layers (Client and TLS Transport) are in the untrusted zone. Everything from the Replay Guard downward runs inside the trusted server process.

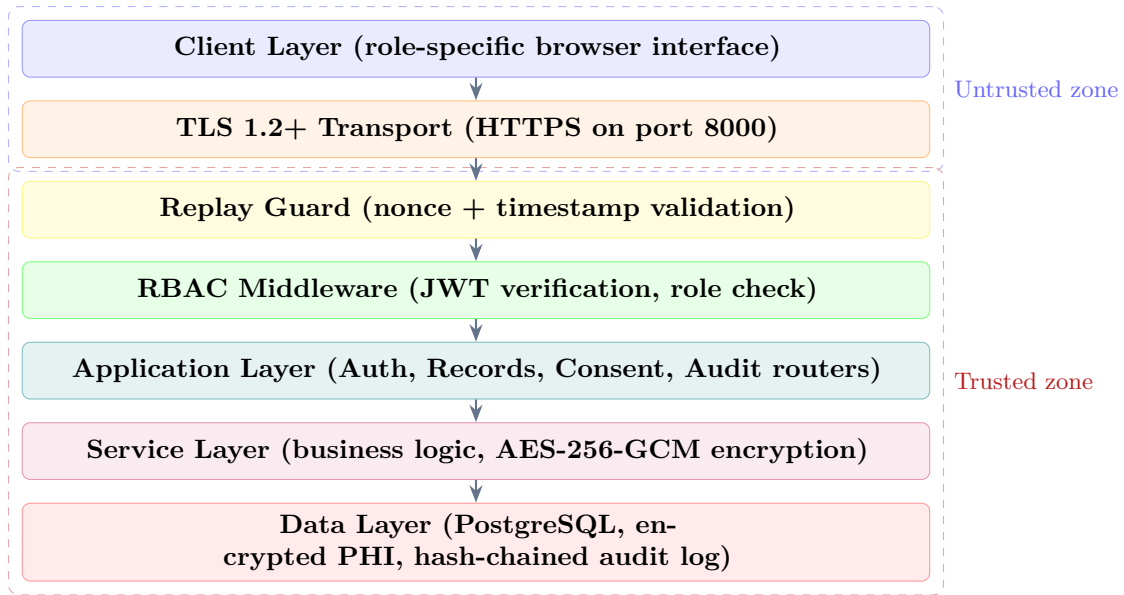


Figure 2: Layered architecture with trust boundaries.

2.2 Security Pipeline

Every request for medical data passes through a fixed sequence of checks before any data is returned. The sequence is:

1. Verify the JWT signature and expiry.
2. Confirm the user role permits the requested action.
3. Check that an active consent grant exists (for doctor access).
4. Decrypt the record using AES-256-GCM on the server.
5. Append an audit log entry with the SHA-256 chain hash.
6. Return the plaintext to the client over the TLS tunnel.

No step can be skipped. Decryption happens entirely on the server; the client never receives a key or ciphertext.

2.3 Data Separation

Encrypted record data (`encrypted_data`, `iv`, `tag`) is stored in PostgreSQL. The decryption key (`RECORD_ENCRYPTION_KEY`) is loaded from an environment variable at startup and never written to the database. A full database dump therefore contains only ciphertext.

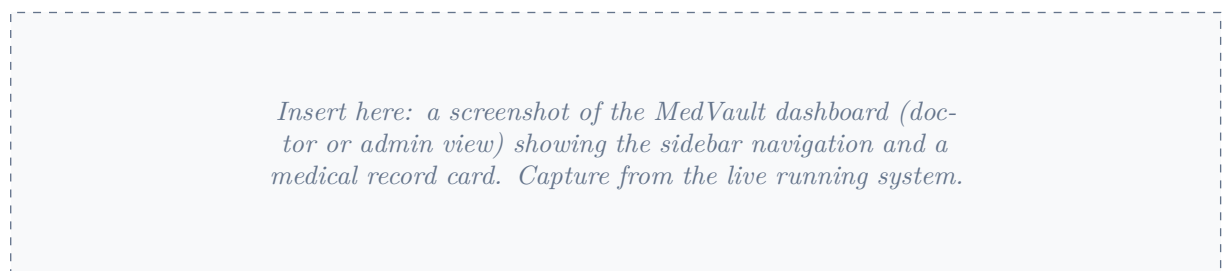


Figure 3: fig:dashboard

3 Threat Model

3.1 Assets

The system protects five categories of assets: patient health information stored in `medical_records`; authentication credentials including password hashes and TOTP secrets; session tokens (access and refresh JWTs); the encryption keys held in environment configuration; and the audit trail itself, whose integrity is a prerequisite for non-repudiation.

3.2 Adversary Classes

Three adversary classes were considered during design.

External attacker. Has network access and may capture HTTP traffic. Goals include stealing tokens, replaying captured requests, and reading PHI in transit.

Malicious insider. Holds valid credentials and may have read access to the database. Goals include accessing records beyond their role, modifying the audit trail, and reading encrypted data after a database export.

Compromised user. A legitimate account holder whose credentials have been stolen or who is subject to social engineering. The main risk is credential reuse and account takeover.

3.3 STRIDE Mapping

Table 1 maps each STRIDE category to the primary risk in this system and the control that addresses it.

Category	Primary risk	Control
Spoofing	Stolen or weak credentials	bcrypt hashing, JWT tokens, TOTP MFA
Tampering	Unauthorised record changes	AES-GCM authentication tag, RBAC
Repudiation	Users deny their actions	Immutable, hash-chained audit log
Disclosure	PHI exposed in a DB breach	AES-256-GCM field encryption, TLS
Insider threat	Over-privileged staff access	Patient-controlled consent, least privilege
Provisioning	Fake or unverified accounts	Admin-only account creation

Table 1: STRIDE threat mapping and controls.

4 Protocol Design

4.1 Identity and Session Protocol

Authentication uses a two-step flow. In the first step, the client submits credentials over HTTPS. The server looks up the account and runs `bcrypt.checkpw` against the stored hash. To prevent timing-based user enumeration, the server always runs a `bcrypt` check even when the email does not exist, using a pre-computed dummy hash. If the password is correct and MFA is not enabled, a full token pair is issued immediately. If MFA is enabled, a short-lived partial JWT (`partial=true`, five-minute expiry) is returned instead.

In the second step, the client submits the partial token together with a six-digit TOTP code. The

server decrypts the stored TOTP secret using AES-256-GCM, computes the expected code for the current 30-second window and the adjacent windows (to tolerate clock drift), and compares using `hmac.compare_digest` to prevent timing attacks. On success, a full access token (15-minute expiry) and a refresh token (7-day expiry, stored as a SHA-256 hash) are issued.

Refresh tokens are rotated on every use. When a refresh request arrives, the old token is revoked before the new pair is issued. A stolen refresh token can therefore only be used once before it becomes invalid.

4.2 Replay Attack Prevention

Every state-changing endpoint requires two additional headers: **X-Nonce** (a UUID) and **X-Timestamp** (ISO-8601 UTC). The Replay Guard middleware checks two conditions before any business logic runs. First, the timestamp must be within five minutes of the server clock. Second, the nonce must not appear in the `nonce_store` table. If either check fails, the request is rejected with HTTP 400 before authentication is attempted. Accepted nonces are stored with a five-minute TTL, matching the timestamp window, so the table does not grow unboundedly.

4.3 Transport Security

All communication runs over TLS 1.2 or higher. The server is started with the `-ssl-certfile` and `-ssl-keyfile` flags passed to Uvicorn, which wraps every TCP connection in a TLS session before handing it to FastAPI. The application layer never sees unencrypted bytes.

For the development and demonstration environment, a self-signed certificate was generated with the following command:

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem
               -days 365 -nodes
               -subj "/C=RW/ST=Kigali/O=MedVault/CN=localhost"
               -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

This generates a 4096-bit RSA key pair and a self-signed X.509 certificate valid for one year. The `-nodes` flag removes the passphrase from the private key so the server can start without manual intervention. The `subjectAltName` extension is required by current browser versions. The certificate file (`cert.pem`) is sent to every client during the TLS handshake. The private key (`key.pem`) stays on the server. For production, the certificate paths in `.env` are updated to point to a CA-signed certificate. No code change is required.

4.4 Access Control and Consent

Role-based access control is enforced by the `require_roles()` dependency applied to every protected endpoint. The dependency decodes the JWT, reads the `role` claim, and raises HTTP 403 if the role is not in the permitted set. Because the role claim is signed inside the JWT, a user cannot change their role without invalidating the signature.

For doctor access to patient records, role alone is not sufficient. The records service calls `_can_read()`, which queries `consent_grants` for an active, non-expired grant between the requesting doctor and the record owner. This check runs inside the service layer, not at the route level, so it cannot be bypassed by a compromised route handler. Figure 4 shows the full sequence.

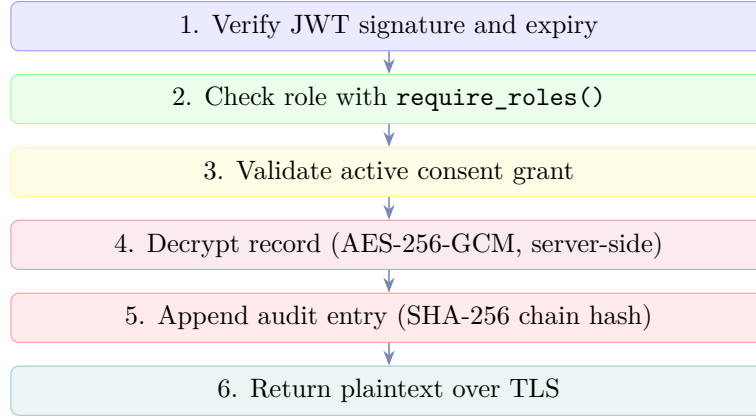


Figure 4: Access control pipeline for a medical record read request.

4.5 Encryption Protocol

Medical records and TOTP secrets are encrypted with AES-256-GCM before being written to the database. For each encryption operation, a fresh 12-byte initialisation vector is generated using `os.urandom(12)`. The ciphertext, IV, and 16-byte authentication tag are stored in separate columns. On retrieval, the tag is verified before any plaintext is returned. A mismatch raises `InvalidTag` and the request fails with HTTP 500.

Two separate 256-bit keys are used: `RECORD_ENCRYPTION_KEY` for medical records and `TOTP_ENCRYPTION_KEY` for TOTP secrets. This limits the impact of a key compromise to one data category. Both keys are loaded from environment variables at startup; the application refuses to start if either is absent or not exactly 32 bytes.

4.6 Audit Chain Protocol

Every sensitive action appends an entry to the audit log. The entry contains the event type, actor ID, resource ID, resource type, client IP, and timestamp. Before the entry is written, the service serialises these fields to a deterministic JSON string (sorted keys, no whitespace) and computes:

$$\text{chain_hash} = \text{SHA-256}(\text{serialised_entry} + \text{previous_chain_hash})$$

The first entry uses a sentinel of 64 zero characters as the previous hash. To verify integrity, `GET /audit/verify` recomputes every hash in sequence and reports the first entry where the stored hash does not match. The database application role has `INSERT` and `SELECT` privileges only on `audit_log`, so the application process cannot issue `UPDATE` or `DELETE` even if it is compromised.

5 Implementation

5.1 Technology Stack

The backend is built with Python 3.11 and FastAPI, using SQLAlchemy 2.x with the `asyncpg` driver for asynchronous PostgreSQL access. Database schema changes are managed with Alembic migrations. The cryptography library provides AES-256-GCM. Password hashing uses `bcrypt`. JWT operations use `PyJWT`. The TOTP module is implemented from scratch using only the Python standard library (`hmac`, `hashlib`, `struct`, `time`, `os`, `base64`), with no third-party dependency, to reduce the supply-chain attack surface.

5.2 Security Components

Table 2 summarises how each security requirement maps to a concrete implementation component.

Component	Implementation
Authentication	Passwords stored as bcrypt hashes. Login issues a 15-minute HS256 JWT with <code>role</code> and <code>sub</code> claims. MFA flow uses a partial JWT (<code>partial=true</code> , 5-min expiry) exchanged for a full token pair after TOTP verification. Refresh tokens are 32-byte random values stored as SHA-256 hashes and rotated on every use.
Access Control	<code>require_roles()</code> FastAPI dependency applied to every protected endpoint. Seven roles: Patient, Doctor, Nurse, Lab_Technician, Front_Desk, Admin, SuperAdmin. Consent check runs inside the service layer for doctor access.
Consent	Doctor initiates a request; patient approves or rejects. Active grants carry an expiry timestamp. Patients can revoke at any time. State transitions: PENDING, ACTIVE, REJECTED, REVOKED, EXPIRED.
Encryption	AES-256-GCM with a random 12-byte IV per operation. Ciphertext, IV, and tag stored in separate columns. Separate keys for records and TOTP secrets. Keys loaded from environment variables; startup fails if absent.
Audit Logging	SHA-256 hash chain. DB role restricted to INSERT and SELECT on <code>audit_log</code> . Patients can verify their own entries via <code>GET /audit/verify</code> . Admins verify the full chain.
Admin Controls	SuperAdmin-only account creation. Security alert dashboard at <code>GET /admin/security-alerts</code> aggregates LOGIN_FAILED, ACCESS_DENIED, and replay-blocked events by account and source IP.
Transport	TLS 1.2+ via Uvicorn <code>-ssl-certfile</code> and <code>-ssl-keyfile</code> . Self-signed certificate for development; CA-signed certificate for production, configured through environment variables only.

Table 2: Security component implementation summary.

5.3 Configuration and Secrets Management

No secret value appears in source code or version-controlled files. All secrets are loaded from a `.env` file at startup. The required variables are: `DATABASE_URL`, `JWT_SECRET_KEY`, `RECORD_ENCRYPTION_KEY`, `TOTP_ENCRYPTION_KEY`, `TLS_CERT_FILE`, and `TLS_KEY_FILE`. The Key Manager validates each key at startup and terminates the process with a descriptive error if any variable is missing or if an encryption key is not exactly 32 bytes. A `.env.example` file with placeholder values is committed to the repository so that new developers know which variables to set.

Insert here: a screenshot of the security alerts dashboard (SuperAdmin view) showing the stat cards for failed logins, access denied events, and replay-blocked requests, together with the failed-login table. Capture from the live system.

Figure 5: fig:alerts

5.4 Limitations

TOTP dependency on a smartphone. TOTP enrollment requires a compatible authenticator application (Google Authenticator, Authy, or similar). Users without a smartphone cannot enable MFA, which reduces the security guarantee for those accounts.

Scope of roles. The seven roles modelled cover the main personnel categories in a Kigali hospital but are not exhaustive. A full deployment would integrate with an existing identity directory such as Active Directory or LDAP.

FHIR integration. The current implementation uses a local PostgreSQL database. Real-world interoperability with existing EMR systems (OpenMRS, Bahmni) requires FHIR R4 API adapters, which are outside the scope of this project.

Rate limiting. Brute-force attempts are recorded in the audit log and visible to administrators through the security alert dashboard, but automatic account logout is not yet implemented.

6 Attack and Defence Demonstrations

Eleven attack scenarios were implemented as runnable Python scripts in the `attacks/` directory. Each script demonstrates the attack, shows the system response, and confirms that the defence is active. Table 3 lists each scenario with its script and the control that blocks it.

#	Attack	Script	Defence
1	Replay attack	01_replay_attack.py	Nonce store rejects duplicate nonce
2	Stale timestamp	02_stale_timestamp.py	5-minute timestamp window
3	Privilege escalation	03_privilege_escalation.py	<code>require_roles()</code> returns 403
4	User enumeration	04_jwt_forgery.py	Identical error for wrong email or password
5	Brute force	05_user_enumeration.py	LOGIN_FAILED events in audit log
6	Audit log tampering	07_audit_tampering.py	SHA-256 hash chain detects modification
7	Unauthorised record access	08_unauthorized_record_access.py	Cross-patient gate in <code>_can_read()</code>
8	Cross-patient access	09_cross_patient_access.py	Ownership check on <code>patient_id</code>
9	Draft record leakage	10_draft_record_leakage.py	Status filter (published only)
10	AES-GCM ciphertext tamper	11_aes_gcm_tampering.py	GCM authentication tag raises <code>InvalidTag</code>
11	Token theft	(auth flow)	15-minute expiry, refresh token rotation

Table 3: Attack scenarios and corresponding defences.

Two scenarios are worth highlighting because they demonstrate properties that are not obvious from the architecture description alone.

AES-GCM ciphertext tampering (scenario 10). An attacker with direct database access flips one byte in the `encrypted_data` column using a PostgreSQL `UPDATE`. When the patient next reads the record, the server attempts decryption and the GCM tag verification fails. The

`cryptography` library raises `InvalidTag` before returning any plaintext. The same attack against AES-CBC would succeed because CBC provides no integrity guarantee.

Audit log tampering (scenario 6). An attacker modifies an audit entry directly in PostgreSQL. The `GET /audit/verify` endpoint recomputes the hash chain from the first entry. Because the modified entry produces a different hash, every subsequent stored hash is also wrong. The endpoint reports the first broken link, including the entry ID and event type. Patients can run the same verification on their own entries.

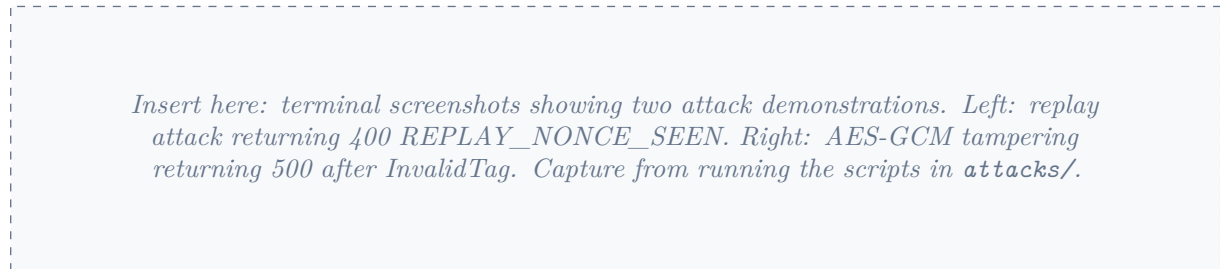


Figure 6: fig:attacks

7 Conclusion

MedVault shows that a meaningful security layer can be added to existing healthcare infrastructure without replacing it. The system addresses the five course deliverables: a layered architecture with documented trust boundaries (Figure 2), a STRIDE threat model (Table 1), a formally described protocol covering authentication, encryption, replay prevention, and audit chaining (Figure 4), a working Python prototype with 20 API endpoints and 12 database tables, and eleven adversarial demonstrations (Table 3).

The combination of TLS at the transport layer, JWT and TOTP at authentication, RBAC and consent at authorisation, AES-256-GCM at storage, and a hash-chained audit log at accountability means that no single compromised component exposes the full system. Each layer provides an independent check that the others do not duplicate.

Tech stack: Python 3.11, FastAPI, PostgreSQL 15, SQLAlchemy 2, AES-256-GCM, bcrypt, RFC 6238 TOTP, TLS 1.2+.